← Learn

# Chunking Strategies for LLM Applications

 Roie Schwaber-Cohen, Arjun Patel

Jun 28, 2025

Share:

Core Components

Jump to section:



## What is chunking?

In the context of building LLM-related applications, **chunking** is the process of breaking down large text into smaller segments called chunks.

It's an essential preprocessing technique that helps optimize the relevance of the content ultimately stored in a vector database. The trick lies in finding chunks that are big enough to contain meaningful information, while small enough to enable performant applications and low latency responses for workloads such as retrieval augmented generation and agentic workflows.

In this post, we'll explore several chunking methods and discuss the tradeoffs needed when choosing a chunking size and method. Finally, we'll give some recommendations for determining the best chunk size and method that will be appropriate for your application.



## Why do we need chunking for our applications?

There are two big reasons why chunking is necessary for any application involving vector databases or LLMs: to ensure embedding models can fit the data into their context windows, and to ensure the chunks themselves contain the information necessary for search.

All embedding models have context windows, which determine the amount of information in tokens that can be processed into a single fixed size vector. Exceeding this context window may means the excess tokens are truncated, or thrown away, before being processed into a vector. This is potentially harmful as important context could be removed from the representation of the text, which prevents it from being surfaced during a search.

Furthermore, it isn't enough just to right-size your data for a model; the resulting chunks must contain information that is relevant to search over. If the chunk

contains a set of sentences that aren't useful without context, they may not be surfaced when querying!

## Chunking's role in semantic search

For example, in semantic search, we index a corpus of documents, with each document containing valuable information on a specific topic. Due to the way embedding models work, those documents will need to be chunked, and similarity is determined by chunk-level comparisons to the input query vector. Then, these similar chunks are returned back to the user. By finding an effective chunking strategy, we can ensure our search results accurately capture the essence of the user's query.

If our chunks are too small or too large, it may lead to imprecise search results or missed opportunities to surface relevant content. As a rule of thumb, if the chunk of text makes sense without the surrounding context to a human, it will make sense to the language model as well. Therefore, finding the optimal chunk size for the documents in the corpus is crucial to ensuring that the search results are accurate and relevant.

## Chunking's role for agentic applications and retrieval-augmented generation

Agents may need access to up-to-date information from databases in order to call tools, make decisions, and respond to user queries. Chunks returned from searches over databases consume context during a session, and ground the agent's responses.

We use the embedded chunks to build the context based on a knowledge base the agent has access to. This context grounds the agent in trusted information.

Similar to how semantic search relies on a good chunking strategy to provide usable outputs, agentic applications need meaningful chunks of information in order to proceed. If an agent is misinformed, or provided information without sufficient context, it may waste tokens generating hallucinations or calling the wrong tools.

# The Role of Chunking for Long Context LLMs

In some cases, like when using o1 or Claude 4 Sonnet with a 200k context window, un-chunked documents may still fit in context. Still, using large chunks may increase latency and cost in downstream responses.

Moreover, long context embedding and LLM models suffer from the lost-in-the-middle problem, where relevant information buried inside long documents is missed, even when included in generation. The solution to this problem is ensuring the optimal amount of information is passed to a downstream LLM, which necessarily reduces latency and ensures quality.

## What should we think about when choosing a chunking strategy?

Several variables play a role in determining the best chunking strategy, and these variables vary depending on the use case. Here are some key aspects to keep in mind:

1. **What kind of data is being chunked?** Are you working with long documents, such as articles or books, or shorter content, like tweets, product descriptions, or chat messages? Small documents may not need to be chunked at all, while larger ones may exhibit certain structure that will inform chunking strategy, such as sub-headers or chapters.
2. **Which embedding model are you using?** Different embedding models have differing capacities for information, especially on specialized domains like code, finance, medical, or legal information. And, the way these models are trained can strongly affect how they perform in practice. After choosing an appropriate model for your domain, be sure to adapt your chunking strategy to align with expected document types the model has been trained on.
3. **What are your expectations for the length and complexity of user queries?** Will they be short and specific or long and complex? This may inform the way you choose to chunk your content as well so that there's a closer correlation between the embedded query and embedded chunks.
4. **How will the retrieved results be utilized within your specific application?** For example, will they be used for semantic search, question answering, retrieval augmented generation, or even an agentic workflow? For example, the amount of information a human may review from a search result may be smaller or larger than what an LLM may need to generate a response. These users determine how your data should be represented within the vector database.

Answering these questions beforehand will allow you to choose a chunking strategy that balances performance and accuracy.

# Embedding short and long content

When we embed content, we can expect distinct behaviors depending on whether the content is short (like sentences) or long (like paragraphs or entire documents).

When a **sentence** is embedded, the resulting vector focuses on the sentence's specific meaning. This could be handy in situations where the vector search is used for (sentence-level) classification, recommendation systems, or applications that allow for searches over shorter summaries before longer documents are processed. The search process is then, finding sentences similar in meaning to query sentences or questions. In cases where sentences themselves are considered individual documents, you wouldn't need to chunk at all!

When a **full paragraph or document** is embedded, the embedding process considers both the overall context and the relationships between the sentences and phrases within the text. This can result in a more comprehensive vector representation that captures the broader meaning and themes of the text. Larger input text sizes, on the other hand, may introduce noise or dilute the significance of individual sentences or phrases, making finding precise matches when querying the index more difficult. These chunks help support use cases such as question-answering, where answers may be a few paragraphs or more. Many modern AI applications work with longer documents, which almost always require chunking.

# Chunking methods

## Fixed-size chunking

This is the most common and straightforward approach to chunking: we simply decide the number of tokens in our chunk, and use this number to break up our documents into fixed size chunks. Usually, this number is the max context window size of the embedding model (such as 1024 for llama-text-embed-v2, or 8196 for text-embedding-3-small). Keep in mind that different embedding models may tokenize text differently, so you will need to estimate token counts accurately.

Fixed-sized chunking will be the best path in most cases, and we recommend starting here and iterating only after determining it insufficient.

## "Content-aware" Chunking

Although fixed-size chunking is quite easy to implement, it can ignore critical structure within documents that can be used to inform relevant chunks. Content-aware chunking refers to strategies that adhere to structure to help inform the meaning of our chunks.

### Simple Sentence and Paragraph splitting

As we mentioned before, some embedding models are optimized for embedding sentence-level content. But sometimes, sentences need to be mined from larger text datasets that aren't preprocessed. In these cases, it's necessary to use sentence chunking, and there are several approaches and tools available to do this:

- **Naive splitting:** The most naive approach would be to split sentences by periods (".), new lines, or white space.
- NLTK: The Natural Language Toolkit (NLTK) is a popular Python library for working with human language data. It provides a trained sentence tokenizer that can split the text into sentences, helping to create more meaningful chunks.
- spaCy: spaCy is another powerful Python library for NLP tasks. It offers a sophisticated sentence segmentation feature that can efficiently divide the text into separate sentences, enabling better context preservation in the resulting chunks.

### Recursive Character Level Chunking

LangChain implements a RecursiveCharacterTextSplitter that tries to split text using separators in a given order. The default behavior of the splitter uses the ["\n\n", "\n", " ", ""] separators to break paragraphs, sentences and words depending on a given chunk size.

This is a great middle ground between always splitting on a specific character and using a more semantic splitter, while also ensuring fixed chunk sizes when possible.

## Document structure-based chunking

When chunking large documents such as PDFs, DOCX, HTML, code snippets, Markdown files and LaTex, specialized chunking methods can help preserve the original structure of the content during chunk creation.

- PDF documents contain loads of headers, text, tables, and other bits and pieces that require preprocessing to chunk. LangChain has some handy utilities to help process these documents, while Pinecone Assistant can chunk and processes these for you
- HTML, from scraped web pages can contain tags (<p> for paragraphs, or <title> for titles) that can inform text to be broken up or identified, like on product pages or blog posts. Roll your own parser, or use LangChain splitters here to process these for chunking
- **Markdown**: Markdown is a lightweight markup language commonly used for formatting text. By recognizing the Markdown syntax (e.g., headings, lists, and code blocks), you can intelligently divide the content based on its structure and hierarchy, resulting in more semantically coherent chunks.
- **LaTex**: LaTeX is a document preparation system and markup language often used for academic papers and technical documents. By parsing the LaTeX commands and environments, you can create chunks that respect the logical organization of the content (e.g., sections, subsections, and equations), leading to more accurate and contextually relevant results.

## Semantic Chunking

A new experimental technique for approaching chunking was first introduced by Greg Kamradt. In his notebook, Kamradt rightfully points to the fact that a global chunking size may be too trivial of a mechanism to take into account the **meaning** of segments within the document. If we use this type of mechanism, we can't know if we're combining segments that have anything to do with one another.

Luckily, if you're building an application with LLMs, you most likely already have the ability to create **embeddings -** and embeddings can be used to extract the semantic meaning present in your data. This semantic analysis can be used to create chunks that are made up of sentences that talk about the same theme or topic.

Semantic chunking involves breaking a document into sentences, grouping each sentence with its surrounding sentences, and generating embeddings for these groups. By comparing the semantic distance between each group and its

predecessor, you can identify where the topic or theme shifts, which defines the chunk boundaries. You can learn more about applying semantic chunking with Pinecone here.

### Contextual Chunking with LLMs

Sometimes, it's not possible to chunk information from a larger complex document without losing the context entirely. This can happen when the documents are many hundreds of pages, change topics frequently, or require understanding from many related portions of the document. Anthropic introduced contextual retrieval in 2024 to help address this problem.

Anthropic prompted a Claude instance with an entire document and it's chunk, in order to generate a contextualized description, which is appended to the chunk and then embedded. The description helps retain the high-level summary meaning of the document to the chunk, which exposes this information to incoming queries. To avoid processing the document each time, it's cached within the prompt for all necessary chunks. You can learn more about contextual retrieval in our video here and our code example here.

## Figuring out the best chunking strategy for your application

Here are some pointers to help decide a strategy if fixed chunking doesn't easily apply to your use case.

- **Selecting a Range of Chunk Sizes** - Once your data is preprocessed, the next step is to choose a range of potential chunk sizes to test. As mentioned previously, the choice should take into account the nature of the content (e.g., short messages or lengthy documents), the embedding model you'll use, and its capabilities (e.g., token limits). The objective is to find a balance between preserving context and maintaining accuracy. Start by exploring a variety of chunk sizes, including smaller chunks (e.g., 128 or 256 tokens) for capturing more granular semantic information and larger chunks (e.g., 512 or 1024 tokens) for retaining more context.
- **Evaluating the Performance of Each Chunk Size** - In order to test various chunk sizes, you can either use multiple indices or a single index with multiple namespaces. With a representative dataset, create the embeddings for the chunk sizes you want to test and save them in your index (or indices). You can then run a series of queries for which you can

evaluate quality, and compare the performance of the various chunk sizes. This is most likely to be an iterative process, where you test different chunk sizes against different queries until you can determine the best-performing chunk size for your content and expected queries.

### Post-processing chunks with chunk expansion

It's important to remember that you aren't entirely married to your chunking strategy. When querying chunked data in a vector database, the retrieved information is typically the top semantically similar chunks given a user query. But users, agents or LLMs may need more surrounding context in order to adequately interpret the chunk.

Chunk expansion is an easy way to post-process chunked data from a database, by retrieving neighboring chunks within a window for each chunk in a retrieved set of chunks. Chunks could be expanded to paragraphs, pages, or even whole documents depending on your use case.

Coupling a chunking strategy with a good chunk expansion on querying can ensure low latency searches without compromising on context.

## Wrapping up

Chunking your content is may appear straightforward in most cases - but it could present some challenges when you start wandering off the beaten path. There's no one-size-fits-all solution to chunking, so what works for one use case may not work for another.

Want to get started experimenting with chunking strategies? Create a free Pinecone account and check out our example notebooks to implement chunking via various applications like semantic search, retrieval augmented generation or agentic applications with Pinecone.

RECOMMENDED FOR YOU

# Further Reading

Learn Jun 12, 2025

## Retrieval-Augmented Generation (RAG)

 Jenna Pederson

Learn Sep 6, 2024

## Vectors and Graphs: Better Together

 Roie Schwaber-Cohen

Blog May 14, 2024

## Build Better RAG Applications with Pinecone and Vectorize

 Anne Colbeck

Pinecone